



PARALLEL AND DISTRIBUTED COMPUTING PROJECT

OPENSTACK INSTALLATION

2nd DELIVERABLE - II

SUBMITTED TO: DR. SONIA SHAHZADI

SUBMITTED BY: FAIZA MARYAM

SECTION: BS-CYS-S24

ROLL NO: 247013

OVERVIEW

This document is the second part of the 2nd deliverable for the OpenStack Installation project. Building upon the base environment setup covered in the 1st deliverable — which included VM Configuration, Network Setup, Chrony, MariaDB, RabbitMQ, Memcached, Etcd, Keystone, and Glance, this document covers the installation and configuration of Placement, Nova, and Neutron, which are essential for resource tracking, compute management, and virtual networking in the OpenStack cloud.

HORIZON

Horizon is the **OpenStack Dashboard service** that provides a **web-based graphical interface (GUI)** for managing OpenStack resources. It allows users and administrators to interact with OpenStack services like Nova (compute), Neutron (networking), Glance (images), and Cinder (storage) without using command-line tools. Through Horizon, users can launch virtual machines, manage networks, upload images, create volumes, and monitor resources easily using a browser. In simple words, Horizon is the front-end dashboard of OpenStack that makes cloud management user-friendly and visual.

STEP 01 Install Package

This shows the terminal output of running `sudo apt install openstack-dashboard` on the controller node. The system is downloading and installing 21 new packages including Django, Horizon, and all related Python dependencies. This package provides the complete web interface for OpenStack management.

```
faiza5@controller:~$ sudo apt install openstack-dashboard
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  openstack-dashboard-common python3-bson python3-bson-ext python3-csscompressor python3-django python3-django-appconf
  python3-django-debreach python3-django-horizon python3-django-openstack-auth python3-django-pyscss python3-gridfs py
  python3-pymongo python3-pymongo-ext python3-pyscss python3-rcssmin python3-rjsmin python3-sqlparse
Suggested packages:
  bpython3 geoip-database-contrib gettext ipython3 libgdal20 libsqlite3-mod-spatialite python-django-doc python3-flup p
  python3-sqlite python-django-appconf-doc python3-calmjs python-django-debreach-doc python-pymongo-doc python-sqlparse
The following NEW packages will be installed:
  openstack-dashboard openstack-dashboard-common python3-bson python3-bson-ext python3-csscompressor python3-django py
  python3-django-compressor python3-django-debreach python3-django-horizon python3-django-openstack-auth python3-django
  python3-heatclient python3-pint python3-pymongo python3-pymongo-ext python3-pyscss python3-rcssmin python3-rjsmin py
0 upgraded, 21 newly installed, 0 to remove and 4 not upgraded.
Need to get 14.4 MB of archives.
After this operation, 97.0 MB of additional disk space will be used.
Do you want to continue? [Y/n] _
```

STEP 02 Configuration

After installation, the Horizon configuration file located at `/etc/openstack-dashboard/local_settings.py` is edited to connect Horizon with the OpenStack services. Key

configurations include setting the OpenStack host to the controller node, configuring the Keystone authentication URL, enabling memcached session storage for better performance, enabling multi-domain support, and setting the correct timezone to Asia/Karachi. These settings ensure Horizon can properly communicate with all OpenStack services.

```
faiza5@controller:~$ sudo nano /etc/openstack-dashboard/local_settings.py_
```

```
COMPRESS_OFFLINE = True
OPENSTACK_HOST = "controller"

OPENSTACK_KEYSTONE_URL = "http://s:5000/identity/v3" % OPENSTACK_HOST
ALLOWED_HOSTS = ['*']

SESSION_ENGINE = 'django.contrib.sessions.backends.cache'

CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.memcached.PyMemcacheCache',
        'LOCATION': 'controller:11211',
    }
}

OPENSTACK_KEYSTONE_MULTIDOMAIN_SUPPORT = True
OPENSTACK_KEYSTONE_DEFAULT_DOMAIN = "Default"
OPENSTACK_KEYSTONE_DEFAULT_ROLE = "user"

TIME_ZONE = "Asia/Karachi"
```

STEP 03 Restart

Both the Apache2 web server and Memcached caching service must be restarted to apply the new settings. The commands `sudo systemctl restart apache2` and `sudo systemctl restart memcached` are executed. Apache2 hosts the Horizon web application while Memcached handles session caching, so restarting both ensures the configuration changes take effect immediately.

```
faiza5@controller:~$ sudo systemctl restart apache2
faiza5@controller:~$ sudo systemctl restart memcached
faiza5@controller:~$ _
```

Verify

The Apache HTTP Server is confirmed as **active (running)** since May 9, 2026. Multiple WSGI worker processes are visible for Horizon, Keystone, and Placement services, confirming the web server is properly hosting all OpenStack dashboard components.

```

• apache2.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
  Active: active (running) since Sat 2026-05-09 07:00:44 UTC; 15min ago
  Docs: https://httpd.apache.org/docs/2.4/
  Process: 2352 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
  Main PID: 2410 (apache2)
  Tasks: 134 (limit: 4543)
  Memory: 906.6M (peak: 945.7M swap: 121.9M swap peak: 121.9M)
  CPU: 1min 52.038s
  CGroup: /system.slice/apache2.service
          └─2410 /usr/sbin/apache2 -k start
             └─2412 "(wsgi:horizon) " -k start
                └─2415 "(wsgi:horizon) " -k start
                   └─2416 "(wsgi:horizon) " -k start
                      └─2417 "(wsgi:keystone-pu" -k start
                         └─2419 "(wsgi:keystone-pu" -k start
                            └─2420 "(wsgi:keystone-pu" -k start
                               └─2421 "(wsgi:keystone-pu" -k start
                                  └─2423 "(wsgi:keystone-pu" -k start
                                     └─2425 "(wsgi:placement-a" -k start
                                        └─2426 "(wsgi:placement-a" -k start
                                           └─2427 "(wsgi:placement-a" -k start
                                              └─2428 "(wsgi:placement-a" -k start
                                                 └─2431 "(wsgi:placement-a" -k start
                                                    └─2432 /usr/sbin/apache2 -k start
                                                       └─2436 /usr/sbin/apache2 -k start

May 09 07:00:44 controller systemd[1]: Starting apache2.service - The Apache HTTP Server...
May 09 07:00:44 controller systemd[1]: Started apache2.service - The Apache HTTP Server.
faiza5@controller:~$

```

Nova compute verify: Three Nova services are all enabled and UP: nova-conductor and nova-scheduler running on the controller node in the internal zone, and nova-compute running on the compute node in the nova zone.

```

faiza_compute@compute:~$ echo -e "\n[libvirt]\nvirt_type=qemu" | sudo tee -a /etc/nova/nova.conf
[libvirt]
virt_type=qemu
faiza_compute@compute:~$ sudo systemctl restart nova-compute
faiza_compute@compute:~$ sudo systemctl status nova-compute
• nova-compute.service - OpenStack Compute
  Loaded: loaded (/usr/lib/systemd/system/nova-compute.service; enabled; preset: enabled)
  Active: active (running) since Sat 2026-05-09 07:28:23 UTC; 10s ago
  Main PID: 15581 (nova-compute)
  Tasks: 2 (limit: 2205)
  Memory: 112.9M (peak: 113.1M)
  CPU: 2.009s
  CGroup: /system.slice/nova-compute.service
          └─15581 /usr/bin/python3 /usr/bin/nova-compute --config-file=/etc/nova/nova.conf --config-file=/etc/nova/nova-compute.conf --log-file=/var/log/nov

May 09 07:28:23 compute systemd[1]: Started nova-compute.service - OpenStack Compute.
May 09 07:28:23 compute nova-compute[15581]: 3 RLock(s) were not greened, to fix this error make sure you run eventlet.monkey_patch() before importing any othe
lines 1-12/12 (END)

```

STEP 04 Check the IP

Before accessing Horizon through a web browser, the controller node's IP address must be identified. The command `ip addr show` displays all network interfaces and their assigned IP addresses. The management network IP address (192.168.107.151 in this case) is the address that will be used in the browser URL to access the Horizon dashboard.

```
faiza5@controller:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:4d:69:f5 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.107.151/24 metric 100 brd 192.168.107.255 scope global dynamic ens33
        valid_lft 951sec preferred_lft 951sec
    inet6 fe80::20c:29ff:fe4d:69f5/64 scope link
        valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:4d:69:ff brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    inet 10.0.0.11/24 brd 10.0.0.255 scope global ens37
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe4d:69ff/64 scope link
        valid_lft forever preferred_lft forever
4: ens38: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:4d:69:09 brd ff:ff:ff:ff:ff:ff
    altname enp2s6
    inet6 fe80::20c:29ff:fe4d:6909/64 scope link
        valid_lft forever preferred_lft forever
faiza5@controller:~$ ls /etc/apache2/conf-enabled/
```

STEP 05 Opening in Browser

This confirms the Horizon dashboard is successfully accessible via the controller node's IP address through the web browser.

192.168.107.151/horizon/auth/login/?next=/horizon/

Canonical
OpenStack Dashboard

Log in

User Name

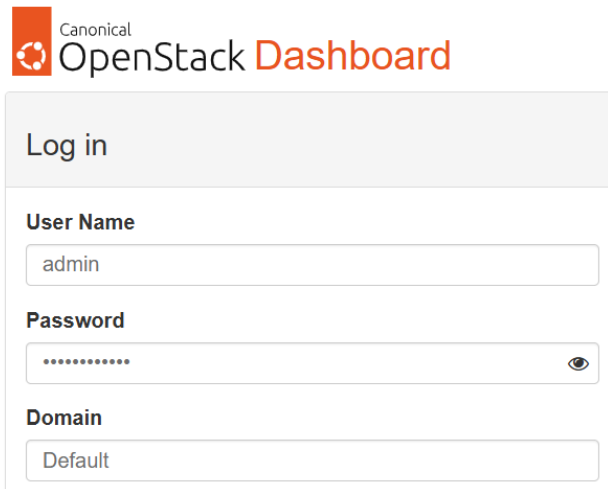
Password

Domain

Sign In

```
faiza5@controller:~$ cat ~/admin-openrc
export OS_PROJECT_DOMAIN_NAME=Default
export OS_USER_DOMAIN_NAME=Default
export OS_PROJECT_NAME=admin
export OS_USERNAME=admin
export OS_PASSWORD=openstack123
export OS_AUTH_URL=http://controller:5000/v3
export OS_IDENTITY_API_VERSION=3
export OS_IMAGE_API_VERSION=2
faiza5@controller:~$ _
```

LOG IN



The image shows the OpenStack Dashboard login interface. At the top, there is a Canonical logo and the text "OpenStack Dashboard". Below this is a "Log in" section with a light gray header. The form contains three fields: "User Name" with the value "admin", "Password" with masked characters and a toggle icon, and "Domain" with the value "Default".

Canonical
OpenStack Dashboard

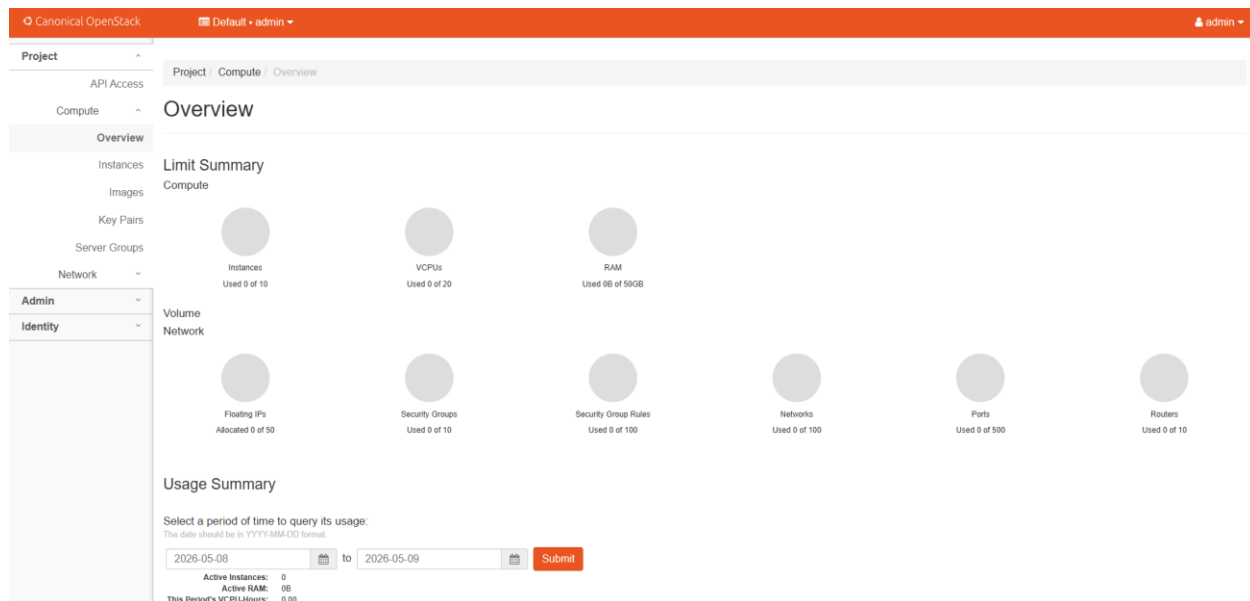
Log in

User Name
admin

Password
.....

Domain
Default

STEP 06 Sign in



STEP 07 Verify

```
faiza5@controller:~$ openstack service list
```

ID	Name	Type
0298d775ea8e419bb0af41a1077cb3e6	placement	placement
dd55df6def0e45b993b48baedc77c181	neutron	network
e54e15ca7a4f469fab934b621455915b	nova	compute
e8945a9fdb204616835bd94cfd208aa6	keystone	identity
f0689b22cfa84d1bbde7256dde3cc614	glance	image
fe3e395b66214fa8a2226800052320f	neutron	network

PROVIDER NETWORK

A provider network is an external network that connects OpenStack instances directly to the physical network infrastructure. It is created by the administrator and maps to a physical network interface on the controller node. The provider network allows instances to receive IP addresses from the physical network's IP range, enabling direct communication with external systems and the internet. In this setup, the provider network uses a flat network type mapped to the physical provider interface.

STEP 01 Creating Network

This shows the result of running openstack network create command. The output table confirms the network named **provider** was successfully created with: network_type=flat, router:external=External, shared=True, and status=ACTIVE.

```
faiza5@controller:~$ openstack network create --share --external --provider-physical-network provider --provider-network-type flat provider
+-----+-----+
| Field | Value |
+-----+-----+
| admin_state_up | UP |
| availability_zone_hints | |
| availability_zones | |
| created_at | 2026-05-09T06:27:59Z |
| description | |
| dns_domain | None |
| id | 5c413a34-620f-4335-8946-aab7e06f9248 |
| ipv4_address_scope | None |
| ipv6_address_scope | None |
| is_default | False |
| is_vlan_qinq | None |
| is_vlan_transparent | None |
| mtu | 1500 |
| name | provider |
| port_security_enabled | True |
| project_id | 207de5ed9b15475b852e0ab54f3437a0 |
| provider:network_type | flat |
| provider:physical_network | provider |
| provider:segmentation_id | None |
| qos_policy_id | None |
| revision_number | 1 |
| router:external | External |
| segments | None |
| shared | True |
| status | ACTIVE |
| subnets | |
| tags | |
| updated_at | 2026-05-09T06:27:59Z |
+-----+-----+
faiza5@controller:~$ _
```

STEP 02 Creating Subnet

The subnet **provider-subnet** was created with: CIDR=192.168.107.0/24, gateway=192.168.107.1, DNS=8.8.8.8, DHCP enabled, and allocation pool from 192.168.107.200 to 192.168.107.250.

```
faiza5@controller:~$ openstack subnet create --network provider --allocation-pool start=192.168.107.200,end=192.168.107.250 --dns-nameserver 8.8.8.8 --gateway 192.168.107.1 --subnet-range 192.168.107.0/24 provider-subnet
+-----+-----+
| Field | Value |
+-----+-----+
| allocation_pools | 192.168.107.200-192.168.107.250 |
| cidr | 192.168.107.0/24 |
| created_at | 2026-05-09T06:29:01Z |
| description | |
| dns_nameservers | 8.8.8.8 |
| dns_publish_fixed_ip | None |
| enable_dhcp | True |
| gateway_ip | 192.168.107.1 |
| host_routes | |
| id | 6e45b1b2-f3bc-4caf-abd0-d88752c75fc5 |
| ip_version | 4 |
| ipv6_address_mode | None |
| ipv6_ra_mode | None |
| name | provider-subnet |
| network_id | 5c413a34-620f-4335-8946-aab7e06f9248 |
| project_id | 207de5ed9b15475b852e0ab54f3437a0 |
| revision_number | 0 |
| router:external | True |
| segment_id | None |
| service_types | |
| subnetpool_id | None |
| tags | |
| updated_at | 2026-05-09T06:29:01Z |
+-----+-----+
faiza5@controller:~$
```


FLAVOUR

A flavour in OpenStack defines the hardware specifications (virtual hardware profile) for a virtual machine instance. It specifies the amount of CPU, RAM, and disk storage that an instance will receive when launched. Flavors allow administrators to standardize instance sizes and control resource consumption. Different flavors can be created for different use cases — small flavors for lightweight applications and larger flavors for resource-intensive workloads.

STEP 01 Creating flavor

The m1.nano flavor is created using the command `openstack flavor create --id 0 --vcpus 1 --ram 64 --disk 1 m1.nano`. This creates the smallest possible flavor with 1 virtual CPU, 64MB of RAM, and 1GB of disk space. The flavor is assigned ID 0 and is made publicly available for all projects. The m1.nano flavor is specifically designed for lightweight test images like CirrOS that require minimal resources.

```
faiza5@controller:~$ openstack flavor create --id 0 --vcpus 1 --ram 64 --disk 1 m1.nano
+-----+
| Field | Value |
+-----+
| OS-FLV-DISABLED:disabled | False |
| OS-FLV-EXT-DATA:ephemeral | 0 |
| description | None |
| disk | 1 |
| id | 0 |
| name | m1.nano |
| os-flavor-access:is_public | True |
| properties |  |
| ram | 64 |
| rxtx_factor | 1.0 |
| swap | 0 |
| vcpus | 1 |
+-----+
faiza5@controller:~$ _
```

STEP 02 Displaying List

After creating the flavor, its existence is verified by running `openstack flavor list`. This command displays all available flavors with their complete specifications including ID, name, RAM, disk size, ephemeral storage, VCPUs, and public availability. Seeing m1.nano in the list confirms it was created successfully and is ready to be used when launching instances.

```
faiza5@controller:~$ openstack flavor list
+-----+
| ID | Name | RAM | Disk | Ephemeral | VCPUs | Is Public |
+-----+
| 0 | m1.nano | 64 | 1 | 0 | 1 | True |
+-----+
faiza5@controller:~$ _
```

KEYPAIR

A key pair in OpenStack is an SSH authentication mechanism consisting of a public key and a private key. The public key is stored in OpenStack and automatically injected into instances when they are launched. The private key is kept by the user and used to establish secure SSH connections to instances without requiring a password. Key pairs provide a more secure method of remote access compared to traditional password-based authentication.

STEP 01 SSH Keygen

Before registering a keypair with OpenStack, an RSA SSH key pair must be generated on the controller node. The command `ssh-keygen -q -N "" -f ~/.ssh/id_rsa` generates the key pair silently (-q flag), with no passphrase (-N ""), and saves the private key to `~/.ssh/id_rsa` and the public key to `~/.ssh/id_rsa.pub`. The public key will then be registered with OpenStack for use with instances.

```
faiza5@controller:~$ ssh-keygen -q -N "" -f ~/.ssh/id_rsa
faiza5@controller:~$ _
```

STEP 02 Generate Key pair

The generated public key is registered with OpenStack using the command `openstack keypair create --public-key ~/.ssh/id_rsa.pub mykey`. This uploads the public key to the Keystone/Nova service and creates a keypair named `mykey`. When an instance is launched with this keypair, the public key is automatically injected into the instance, allowing SSH access using the corresponding private key.

```
faiza5@controller:~$ openstack keypair create --public-key ~/.ssh/id_rsa.pub mykey
+-----+
| Field      | Value                                     |
+-----+-----+
| created_at | None                                     |
| fingerprint | 56:92:c3:29:08:2e:c0:d9:44:fa:27:f3:95:2a:66:54 |
| id         | mykey                                   |
| is_deleted  | None                                     |
| name       | mykey                                   |
| type       | ssh                                     |
| user_id    | ff28630c609347248eee27499adf92d7       |
+-----+-----+
faiza5@controller:~$
```

STEP 03 Displaying Key pair list

The registered key pair is verified by running `openstack keypair list`. This command displays all key pairs available in the system including their name, fingerprint, and type. Seeing `mykey` with type `SSH` in the list confirms the key pair was successfully registered and is ready to be assigned to instances during launch.

```
faiza5@controller:~$ openstack keypair list
+-----+-----+-----+
| Name | Fingerprint | Type |
+-----+-----+-----+
| mykey | 56:92:c3:29:08:2e:c0:d9:44:fa:27:f3:95:2a:66:54 | ssh |
+-----+-----+-----+
faiza5@controller:~$
```

SECURITY GROUP AND RULES

Security groups in OpenStack act as virtual firewalls that control incoming and outgoing network traffic for instances. Each instance is assigned one or more security groups, and the rules within those groups determine what network traffic is allowed. By default, OpenStack blocks all incoming traffic, so specific rules must be added to allow desired connections such as SSH (port 22) for remote access and ICMP for ping functionality. Security groups provide network-level security for all instances in the project.

STEP 01 Create group rules

Two important security group rules are added to the default security group. First, an ICMP rule is created using `openstack security group rule create --proto icmp default` which allows ping traffic to reach instances from any IP address. Second, a TCP rule for port 22 is created using `openstack security group rule create --proto tcp --dst-port 22 default` which allows SSH connections to instances. These rules are essential for basic instance connectivity and remote management.

```
faiza5@controller:~$ openstack security group rule create --proto tcp --dst-port 22 default
+-----+-----+
| Field | Value |
+-----+-----+
| belongs_to_default_sg | True |
| created_at | 2026-05-09T06:32:22Z |
| description | |
| direction | ingress |
| ether_type | IPv4 |
| id | 1b263d63-c946-4c3c-91b1-af35accf3dab |
| name | None |
| normalized_cidr | 0.0.0.0/0 |
| port_range_max | 22 |
| port_range_min | 22 |
| project_id | 207de5ed9b15475b852e0ab54f3437a0 |
| protocol | tcp |
| remote_address_group_id | None |
| remote_group_id | None |
| remote_ip_prefix | 0.0.0.0/0 |
| revision_number | 0 |
| security_group_id | 4a8bf4f3-7179-4131-9a1c-f367bb34278e |
| tags | [] |
| updated_at | 2026-05-09T06:32:22Z |
+-----+-----+
faiza5@controller:~$ _
```

```
faiza5@controller:~$ openstack security group rule create --proto icmp default
```

Field	Value
belongs_to_default_sg	True
created_at	2026-05-09T06:31:44Z
description	
direction	ingress
ether_type	IPv4
id	a5b39afb-cf52-4619-b754-fc32fd280a35
name	None
normalized_cidr	0.0.0.0/0
port_range_max	None
port_range_min	None
project_id	207de5ed9b15475b852e0ab54f3437a0
protocol	icmp
remote_address_group_id	None
remote_group_id	None
remote_ip_prefix	0.0.0.0/0
revision_number	0
security_group_id	4a8bf4f3-7179-4131-9a1c-f367bb34278e
tags	[]
updated_at	2026-05-09T06:31:44Z

STEP 02 Displaying group list

The security group is verified by running `openstack security group list`. This command displays all security groups in the current project including their UUID, name, description, and project ID. The output confirms that the default security group exists and is properly associated with the admin project.

```
faiza5@controller:~$ openstack security group list
```

ID	Name	Description	Project	Tags
4a8bf4f3-7179-4131-9a1c-f367bb34278e	default	Default security group	207de5ed9b15475b852e0ab54f3437a0	[]

STEP 03 Displaying Default group rule list

All rules within the default security group are listed using `openstack security group rule list default`. This displays all six rules including IPv4 and IPv6 egress rules, TCP port 22 ingress rule for SSH, ICMP ingress rule for ping, and self-referencing ingress rules. This verification confirms that all necessary network access rules are properly configured.

```
faiza5@controller:~$ openstack security group rule list default
```

ID	IP Protocol	Ethertype	IP Range	Port Range	Direction	Remote Security Group	Remote Address Group
00522f19-4481-475b-adca-036d0802bb81	None	IPv6	::/0		ingress	4a8bf4f3-7179-4131-9a1c-f367bb34278e	None
1b263d63-c946-4c3c-91b1-af35accf3dab	tcp	IPv4	0.0.0.0/0	22:22	ingress	None	None
5505e9d8-1c05-4345-9380-437f60ecd915	None	IPv6	::/0		egress	None	None
75ddb12a-e2a8-4700-9641-7e45a9f1d3fe	None	IPv4	0.0.0.0/0		ingress	4a8bf4f3-7179-4131-9a1c-f367bb34278e	None
a5b39afb-cf52-4619-b754-fc32fd280a35	icmp	IPv4	0.0.0.0/0		ingress	None	None
e2785860-e600-470d-97f1-d9bd08205b84	None	IPv4	0.0.0.0/0		egress	None	None

```
faiza5@controller:~$ _
```

STEP 04 Image list

The available images are verified by running `openstack image list`. This confirms that the CirrOS image is present in the Glance image service with status active. The CirrOS image is a minimal Linux distribution specifically designed for testing OpenStack deployments. Its small size (around 20MB) makes it ideal for quickly launching test instances.

```
faiza5@controller:~$ openstack image list
+-----+-----+-----+
| ID                | Name   | Status |
+-----+-----+-----+
| c294efd3-3c35-40ef-8f91-224be2752a55 | cirros | active |
+-----+-----+-----+
faiza5@controller:~$ _
```

INSTANCE

An instance in OpenStack is a virtual machine that runs on the compute node. Launching an instance requires specifying several parameters: a flavor (hardware specifications), an image (operating system), a network (connectivity), a security group (firewall rules), and a keypair (SSH access). Once launched, the instance runs as an isolated virtual machine on the compute node and can be accessed, managed, and monitored through either the CLI or the Horizon dashboard.

STEP 01 Creating instance

```
faiza5@controller:~$ openstack server create --flavor m1.nano --image cirros --nic net-id=5c413a34-620f-4335-8946-aab7e06f9248 --security-group default --key-name mykey provider-instance
```

STEP 02 Instance Details

The detailed status of the newly created instance is checked using `openstack server show`. This command displays all properties of the instance including its name, image, flavor, assigned IP address, keypair, security groups, current status, power state, and creation timestamp. This step verifies that all the specified parameters were correctly applied to the instance.

```

OS-SRV-USG:launched_at      | None
OS-SRV-USG:terminated_at    | None
accessIPv4                   | None
accessIPv6                   | None
addresses                    | N/A
adminPass                    | j7bfNrdaXfR4
config_drive                 | None
created                      | 2026-05-10T04:56:29Z
description                  | None
flavor                       | description=, disk='1', ephemeral='0', , id='m1.nano', is_disabled=, is_public=
                             | original_name='m1.nano', ram='64', rxtx_factor=, swap='0', vcpus='1'
hostId                       | None
host_status                  | None
id                           | 4a62b142-3b77-4ccf-b426-af229b839961
image                        | cirros (c294efd3-3c35-40ef-8f91-224be2752a55)
key_name                     | mykey
locked                       | None
locked_reason                | None
name                         | FTFirstinstance
pinned_availability_zone     | None
progress                    | None
project_id                   | 207de5ed9b15475b852e0ab54f3437a0
properties                   | None
security_groups              | name='default'
server_groups                | None
status                       | BUILD
tags                         |
trusted_image_certificates   | None
updated                      | 2026-05-10T04:56:29Z
user_id                      | ff28630c609347248eee27499adf92d7
volumes_attached            |

```

STEP 03 Compute service list

Before and after launching an instance, the compute services are verified using openstack compute service list. This confirms that nova-conductor, nova-scheduler, and nova-compute are all running and healthy. Having all compute services active and enabled is essential for instances to be successfully scheduled, built, and run on the compute node.

```

faiza5@controller:~$ openstack compute service list
+-----+-----+-----+-----+-----+-----+-----+
| ID | Binary | Host | Zone | Status | State | Updated At |
+-----+-----+-----+-----+-----+-----+-----+
| eb1f8f6c-92e8-4a00-b942-8addc4967f2f | nova-conductor | controller | internal | enabled | up | 2026-05-09T08:14:43.000000 |
| 8e478b46-5bed-4595-8051-acd1309e12dc | nova-scheduler | controller | internal | enabled | up | 2026-05-09T08:14:43.000000 |
| 0a851d64-330e-4168-bd8e-7a87959278ca | nova-compute | compute | nova | enabled | up | 2026-05-09T08:14:44.000000 |
+-----+-----+-----+-----+-----+-----+-----+
faiza5@controller:~$

```

OPEN IN BROWSER

STEP 01 Login

The Horizon dashboard is accessed via browser to manage and verify the OpenStack deployment graphically. The administrator logs in using username admin, password openstack123, and domain Default. The graphical interface provides a more visual and user-friendly way to manage instances, networks, images, and other OpenStack resources compared to the command line.

Canonical

OpenStack Dashboard

Log in

User Name

admin

Password

openstack123

Domain

default

Sign In

Interface of log in

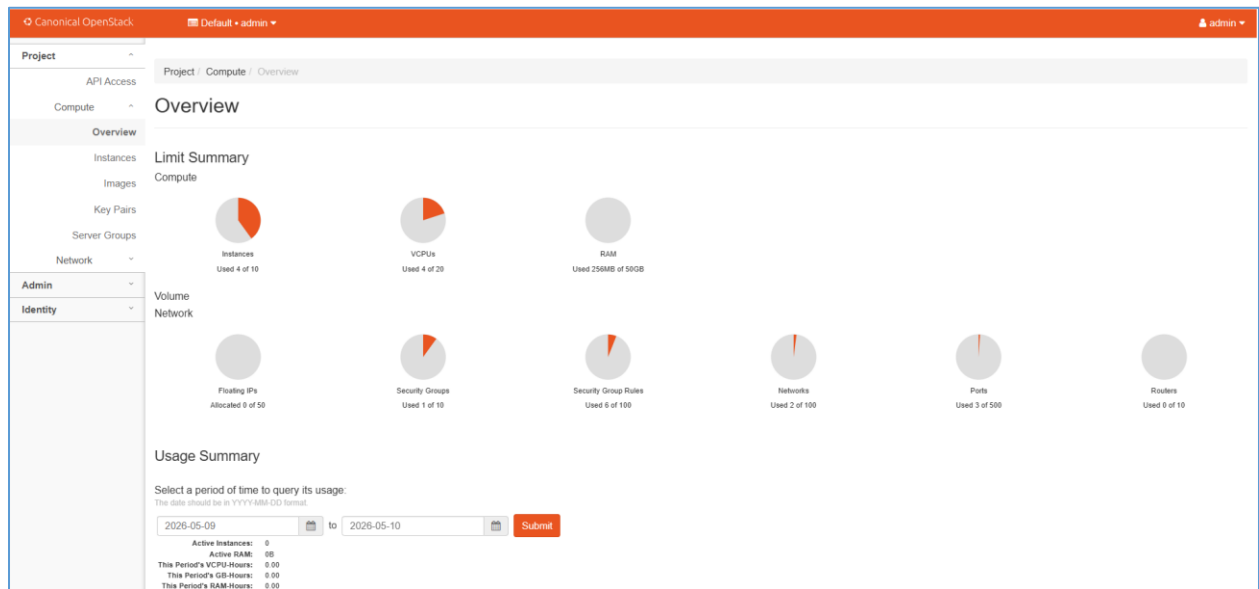


Image list

Images

Click here for filters or full text search.

Create Image

Delete Images

Displaying 1 item

Owner	Name ^	Type	Status	Visibility	Protected	Disk Format	Size
admin	cirros	Image	Active	Public	No	QCOW2	20.44 MB

Displaying 1 item

Key Pair

Key Pairs

Q Click here for filters or full text search.

+ Create Key Pair

Import Public Key

Delete Key Pairs

Displaying 1 item

☐	Name ^	Type	Fingerprint	
☐	> mykey	ssh	56:92:c3:29:08:2e:c0:d9:44:fa:27:f3:95:2a:66:54	Delete Key Pair

Displaying 1 item

Networks

Project / Network / Networks

Networks

Name =

Filter

+ Create Network

Delete Networks

Displaying 2 items

☐	Name	Subnets Associated	Shared	External	Status	Admin State	Availability Zones	Actions
☐	provider	provider-subnet 192.168.107.0/24	Yes	Yes	Active	True	nova	Edit Network
☐	private	private-subnet 192.168.1.0/24	No	No	Active	True	nova	Edit Network

Displaying 2 items

Availability Zones

Availability Zones

Filter

Displaying 2 items

Availability Zone Name	Hosts	Available
internal	• controller (Services Up)	Yes
nova	• compute (Services Up)	Yes

Displaying 2 items

Flavour

Flavours

Filter

+ Create Flavour

Delete Flavours

Displaying 1 item

☐	Flavor Name	VCPUs	RAM	Root Disk	Ephemeral Disk	Swap Disk	RX/TX factor	ID	Public	Metadata	Actions
☐	m1.nano	1	64MB	1GB	0GB	0MB	1.0	0	Yes	No	Update Metadata

Role Based Policies

RBAC Policies

Create RBAC PolicyDelete RBAC Policies

Displaying 2 items

☐	Project	ID	Object Type	Object	Target Project	Actions
☐	admin	6e5ced3c-da0a-462e-b3ac-f76d60a8847d	network	provider	*	Edit Policy
☐	admin	9249829e-5add-45f1-9d81-db533e07c6f0	network	provider	*	Edit Policy

Displaying 2 items

Default

Defaults

Compute Quotas	Network Quotas
Filter	
Displaying 10 items	
Quota Name	Limit
VCPUs	20
Injected File Content (B)	10240
Length of Injected File Path	255
Injected Files	5
Instances	10
Key Pairs	100
Metadata Items	128
RAM (MB)	51200
Server Group Members	10
Server Groups	10
Displaying 10 items	

System Information

System Information			
Services	Compute Services	Network Agents	
Displaying 5 items			
Name	Service	Region	Endpoints
placement	placement	RegionOne	Admin http://controller:8778 Internal http://controller:8778 Public http://controller:8778
neutron	network	RegionOne	Admin http://controller:9696 Internal http://controller:9696 Public http://controller:9696
nova	compute	RegionOne	Admin http://controller:8774/v2.1 Internal http://controller:8774/v2.1 Public http://controller:8774/v2.1
keystone	identity	RegionOne	Admin http://controller:5000/v3/ Internal http://controller:5000/v3/ Public http://controller:5000/v3/
glance	image	RegionOne	Admin http://controller:9292 Internal http://controller:9292 Public http://controller:9292
Displaying 5 items			

All users

Users							
User Name = Filter + Create User Delete Users							
Displaying 5 items							
☐	User Name	Description	Email	User ID	Enabled	Domain Name	Actions
☐	glance	-		0802870749304b048ba40bf8379ee3f9	Yes	Default	Edit Delete
☐	neutron	-		1023ae073e7a42959edeaf8970113d14	Yes	Default	Edit Delete
☐	placement	-		2bf3cb439b84264812c8d68347898ed	Yes	Default	Edit Delete
☐	nova	-		e74a2c4f278a423d9de195eca4db1adc	Yes	Default	Edit Delete
☐	admin	-		ff28630c609347248eee27499adf92d7	Yes	Default	Edit Delete
Displaying 5 items							

STEP 02 Creating Instance

The instance named FTFirstinstance is created using the openstack server create command with all required parameters: flavor=m1.nano, image=cirros, network connected using the provider network ID, security-group=default, and key-name=mykey. Immediately after execution, the instance enters the BUILD state as OpenStack schedules and provisions the virtual machine on the compute node.

Instance Name *

FTFirstInstance

Availability Zone

nova

Count *

1

Total Instances (10 Max)

10%

0 Current Usage

1 Added

9 Remaining

Instance source is the template used to create an instance. You can use an image, a snapshot of an instance (image snapshot), a volume or a volume snapshot (if enabled). You can also choose to use persistent storage by creating a new volume.

Select Boot Source

Image

Allocated

Displaying 1 item

Name	Updated	Size	Format	Visibility
> cirros	4/19/26 9:55 AM	20.44 MB	QCOW2	Public

Displaying 1 item

▼ Allocated 1

Displaying 1 item

Network	Subnets Associated	Shared	Admin State	Status
> provider	provider-subnet	No	Up	Active

Displaying 1 item

▼ Allocated 1

Displaying 1 item	
Name	Description
➤ default	Default security group
Displaying 1 item	

Allocated

Displaying 1 item	
Name	Type
➤ mykey	ssh
Displaying 1 item	

Started Scheduling

☐	FTFirstInstance	cirros	192.168.107.229	m1.nano	mykey	Build	🔊	nova	Spawning	No State	0 minutes	Associate Floating IP
Displaying 3 items												

STATUS = ACTIVE

Displaying 1 item												
☐	Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions	
☐	FTFirstInstance	cirros	provider 192.168.107.209 private 192.168.1.153	m1.nano	mykey	Active	🔊 nova	None	Running	1 minute	Create Snapshot	▼
Displaying 1 item												

Opening console

```
faiza5@controller:~$ openstack console url show FTFirstinstance
```

```
+-----+
| Field | Value |
+-----+
| protocol | vnc |
| type | novnc |
| url | http://controller:6080/vnc_auto.html?path=%3Ftoken%3De5262533-fe7e-41e7-91b1-6eb1dfce8fb9 |
+-----+
faiza5@controller:~$ _
```

ASSOCIATE FLOATING IP

Time since created	Actions
--------------------	---------

0 minutes

Create Snapshot

Manage Floating IP Associations

☐	FTFirstInstance	cirros	Floating IPs:	m1.tiny
			•	172.16.8.134

Displaying 1 item

OPEN NOVNC

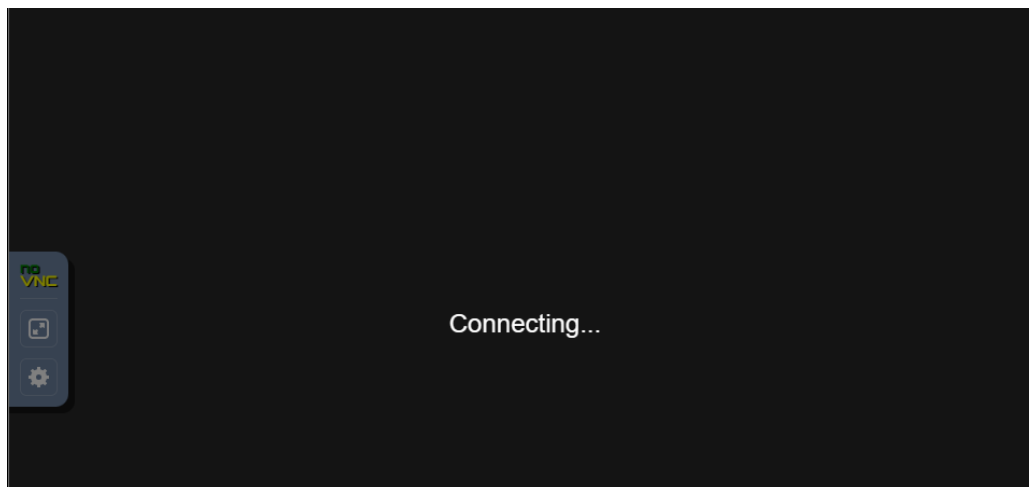
The noVNC web-based VNC client is used to access the console of the running OpenStack instance (FTFirstinstance) directly through the browser. The console URL is generated using the command `openstack console url show FTFirstinstance` on the controller node. This URL contains a unique token that provides secure, time-limited access to the instance's graphical console without requiring SSH or any additional client software.

```
+-----+
| Field | Value |
+-----+
| protocol | vnc |
| type | novnc |
| url | http://controller:6080/vnc_auto.html?path=%3Ftoken%3De5262533-fe7e-41e7-91b1-6eb1dfce8fb9 |
+-----+
faiza5@controller:~$ _
```

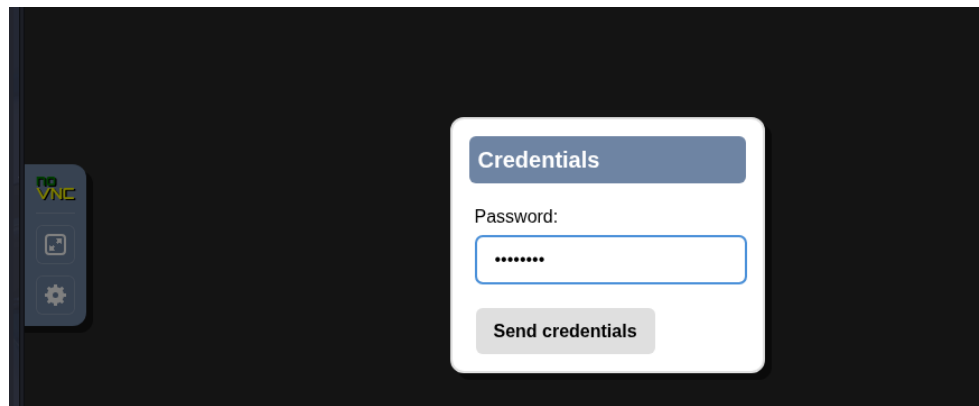
OPEN IN BROWSER

http://192.168.107.201:6080/vnc_auto.html?path=%3Ftoken%3De5262533-fe7e-41e7-91b1-6eb1dfce8fb9

Interface after opening in browser



Entering Credentials



The console output displays the CirrOS operating system kernel boot messages, confirming that the virtual machine instance is successfully running on the compute node.

Login as: **cirros**

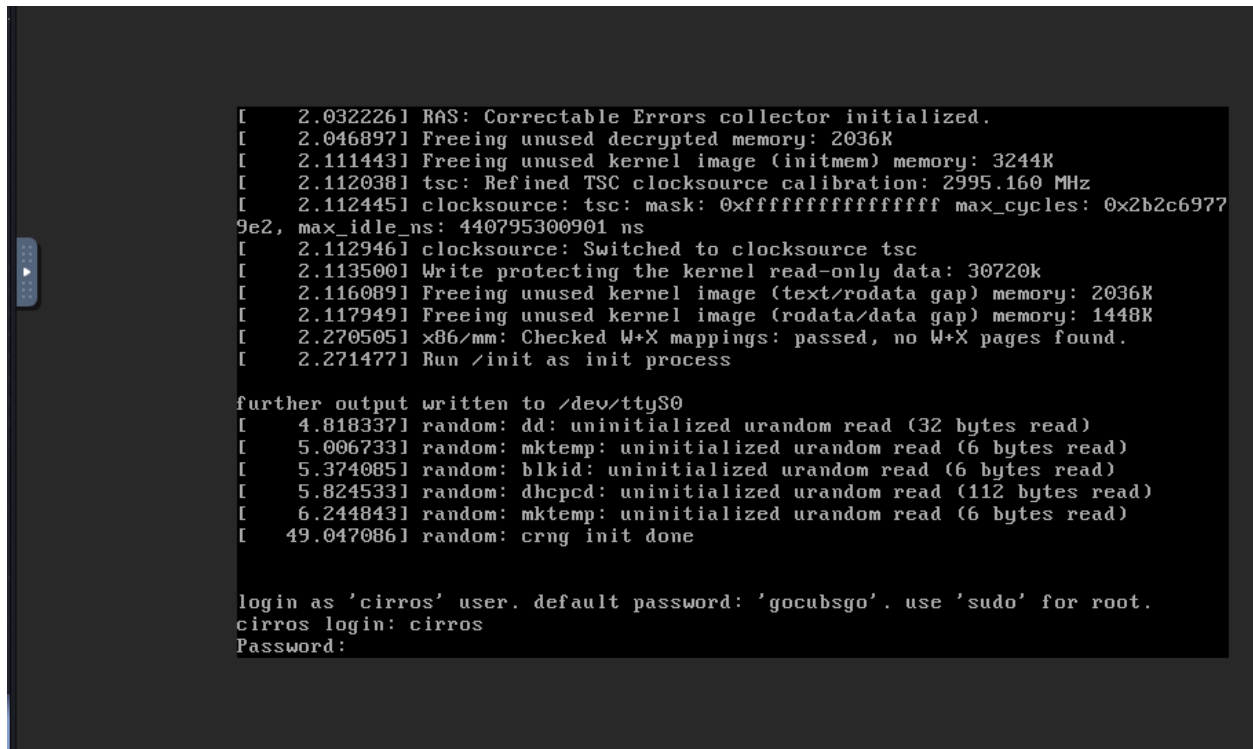
Password: **gocubsgo**

```
[ 2.030156] powernow_k8: Power state transitions not supported
[ 2.032226] RAS: Correctable Errors collector initialized.
[ 2.046897] Freeing unused decrypted memory: 2036K
[ 2.111443] Freeing unused kernel image (initmem) memory: 3244K
[ 2.112038] tsc: Refined TSC clocksource calibration: 2995.160 MHz
[ 2.112445] clocksource: tsc: mask: 0xffffffffffffffff max_cycles: 0x2b2c6977
9e2, max_idle_ns: 440795300901 ns
[ 2.112946] clocksource: Switched to clocksource tsc
[ 2.113500] Write protecting the kernel read-only data: 30720k
[ 2.116089] Freeing unused kernel image (text/rodata gap) memory: 2036K
[ 2.117949] Freeing unused kernel image (rodata/data gap) memory: 1448K
[ 2.270505] x86/mm: Checked W&X mappings: passed, no W&X pages found.
[ 2.271477] Run /init as init process

further output written to /dev/ttyS0
[ 4.818337] random: dd: uninitialized urandom read (32 bytes read)
[ 5.006733] random: mktemp: uninitialized urandom read (6 bytes read)
[ 5.374085] random: blkid: uninitialized urandom read (6 bytes read)
[ 5.824533] random: dhcpcd: uninitialized urandom read (112 bytes read)
[ 6.244843] random: mktemp: uninitialized urandom read (6 bytes read)
[ 49.047086] random: crng init done

login as 'cirros' user. default password: 'gocubsgo'. use 'sudo' for root.
cirros login: _
```

The noVNC console shows the CirrOS instance successfully booted and displaying the login prompt, confirming the virtual machine is fully operational and accessible through the browser-based console.

A screenshot of a noVNC console window. The window has a dark background with a light gray sidebar on the left containing a play button icon. The main area displays white text representing system boot logs. The logs include messages about RAS initialization, memory freeing, kernel image freeing, TSC calibration, clocksource switching, write protecting kernel data, and x86/mm mappings. After the boot logs, there is a message indicating that further output is written to /dev/ttyS0, followed by a series of random seed initialization messages. At the bottom, a login prompt appears for the 'cirros' user with the default password 'gocubsgo'.

```
[ 2.032226] RAS: Correctable Errors collector initialized.
[ 2.046897] Freeing unused decrypted memory: 2036K
[ 2.111443] Freeing unused kernel image (initmem) memory: 3244K
[ 2.112038] tsc: Refined TSC clocksource calibration: 2995.160 MHz
[ 2.112445] clocksource: tsc: mask: 0xffffffffffffffff max_cycles: 0x2b2c6977
9e2, max_idle_ns: 440795300901 ns
[ 2.112946] clocksource: Switched to clocksource tsc
[ 2.113500] Write protecting the kernel read-only data: 30720k
[ 2.116089] Freeing unused kernel image (text/rodata gap) memory: 2036K
[ 2.117949] Freeing unused kernel image (rodata/data gap) memory: 1448K
[ 2.270505] x86/mm: Checked W&X mappings: passed, no W&X pages found.
[ 2.271477] Run /init as init process

further output written to /dev/ttyS0
[ 4.818337] random: dd: uninitialized urandom read (32 bytes read)
[ 5.006733] random: mktemp: uninitialized urandom read (6 bytes read)
[ 5.374085] random: blkid: uninitialized urandom read (6 bytes read)
[ 5.824533] random: dhcpcd: uninitialized urandom read (112 bytes read)
[ 6.244843] random: mktemp: uninitialized urandom read (6 bytes read)
[ 49.047086] random: crng init done

login as 'cirros' user. default password: 'gocubsgo'. use 'sudo' for root.
cirros login: cirros
Password:
```

CONCLUSION

This project successfully demonstrates the complete installation and configuration of an OpenStack cloud computing environment from scratch across a controller and compute node. The deployment covered all essential components including VM configuration, netplan, network setup, time synchronization using Chrony, and installation of supporting services such as MariaDB, RabbitMQ, Memcached, and Etcd. Core OpenStack services were configured in sequence — Keystone, Glance, Placement, Nova, Neutron, and Horizon — followed by provider network creation, flavor definition, keypair generation, and security group configuration. Finally, a CirrOS instance was successfully launched and accessed through the noVNC browser-based console, confirming that the entire OpenStack cloud infrastructure is fully operational and functioning as expected.

